

user

学习java中级开发的技术栈内容有哪些？

assistant

学习 Java 中级开发，意味着你已经掌握了 Java 基础（如语法、面向对象、集合、异常处理、IO 等），接下来需要深入理解更复杂的编程思想、框架、工具和系统设计能力。以下是 Java 中级开发常见的技术栈内容，按模块分类整理：

一、核心 Java 进阶

- 多线程与并发编程
 - Thread、Runnable、Callable
 - 线程池（ExecutorService、ThreadPoolExecutor）
 - 并发工具类（CountDownLatch、CyclicBarrier、Semaphore、ReentrantLock、ReadWriteLock）
 - volatile、synchronized、CAS 原理
 - JUC（java.util.concurrent）包常用类
 - 死锁、活锁、线程安全问题排查
- JVM 基础
 - 内存结构（堆、栈、方法区、元空间等）
 - 垃圾回收机制（GC 算法、常见 GC 器如 G1、CMS）
 - 类加载机制（双亲委派模型）
 - JVM 调优基础（jstat、jstack、jmap、VisualVM）
- Java 新特性（至少掌握到 Java 11）
 - Lambda 表达式、Stream API
 - Optional、新的日期时间 API（LocalDateTime 等）
 - 模块化（Java 9+）
 - var 关键字（Java 10+）

二、数据库与持久层

- SQL 与关系型数据库
 - 熟练编写复杂 SQL（JOIN、子查询、窗口函数）
 - 索引原理与优化
 - 事务 ACID、隔离级别、MVCC

- MySQL/PostgreSQL 常用操作与调优
- ORM 框架
 - MyBatis：动态 SQL、缓存机制、插件开发
 - Hibernate/JPA（了解即可，国内 MyBatis 更主流）
- 连接池
 - HikariCP、Druid（监控、SQL 防火墙）

三、主流框架（Spring 生态）

- Spring Framework
 - IoC / DI 原理
 - AOP（动态代理、@Aspect）
 - Bean 生命周期、作用域
 - 事务管理（@Transactional 原理）
- Spring Boot
 - 自动配置原理（@EnableAutoConfiguration）
 - Starter 机制
 - 配置文件（application.yml / properties）
 - Actuator 监控
 - 自定义 Starter（加分项）
- Spring MVC
 - 请求处理流程
 - 拦截器、异常统一处理
 - 参数校验（Bean Validation）

四、中间件与分布式基础

- 缓存
 - Redis：数据类型、持久化、集群、缓存穿透/击穿/雪崩
 - 缓存与数据库一致性策略
- 消息队列
 - RabbitMQ / Kafka / RocketMQ（至少掌握一种）
 - 消息可靠性、顺序性、幂等性
- 微服务基础（可选但推荐）

- Spring Cloud Alibaba (Nacos、Sentinel、Seata)
- 服务注册发现、配置中心、熔断限流

五、开发工具与工程能力

- 构建工具
 - Maven / Gradle (依赖管理、生命周期)
- 版本控制
 - Git (分支管理、合并冲突、rebase vs merge)
- 测试
 - 单元测试 (JUnit 5、Mockito)
 - 集成测试
- 日志框架
 - SLF4J + Logback / Log4j2
- API 文档
 - Swagger / OpenAPI

六、系统设计与编码规范

- 设计模式
 - 单例、工厂、策略、模板、观察者、责任链等常用模式
- 代码质量
 - 遵循 Clean Code 原则
 - 使用 SonarQube 或 Checkstyle
- RESTful API 设计规范
 - HTTP 方法语义、状态码、资源命名
- 安全性基础
 - XSS、CSRF、SQL 注入防范
 - JWT、OAuth2 基础概念

学习建议：

1. 动手实践：搭建项目整合 Spring Boot + MyBatis + Redis + MQ。
2. 阅读源码：从 Spring Boot 自动配置、MyBatis 执行流程入手。
3. 刷题巩固：LeetCode 中等难度 + 并发编程实战题。
4. 参与开源或模拟项目：如电商后台、博客系统、秒杀系统。

如果你有特定方向（如后端服务、大数据、金融系统等），技术栈侧重点会有所不同。欢迎补充你的目标场景，我可以进一步细化建议。

user

好的，从求职的角度来说，帮我整理下相关的常见面试题，和详细的改怎么回答

assistant

当然可以！从 求职 Java 中级开发岗位 的角度出发，面试官通常会围绕 基础知识、框架原理、系统设计、编码能力、项目经验 等维度提问。下面我为你整理一份 高频面试题清单 + 回答思路与示例答案，帮助你系统准备。

一、Java 核心（重点考察深度）

1. HashMap 的底层原理？为什么线程不安全？

回答思路：

- 结构：JDK 8 后是「数组 + 链表/红黑树」
- 扩容机制：负载因子 0.75，扩容为 2 倍，rehash
- 线程不安全：多线程 put 可能导致环形链表（JDK 7）或数据覆盖（JDK 8）
- 替代方案：ConcurrentHashMap（分段锁 / CAS + synchronized）

✅ 示例回答：

HashMap 在 JDK 8 中底层使用数组 + 链表（超过 8 个转红黑树）。当多个线程同时执行 put 操作且触发扩容时，JDK 7 中可能出现“环形链表”导致死循环；JDK 8 虽然修复了该问题，但仍可能因并发写入导致数据丢失。因此它是线程不安全的。若需线程安全，应使用 ConcurrentHashMap，它通过 CAS 和 synchronized 锁住桶头节点来保证并发安全。

2. synchronized 和 ReentrantLock 的区别？

回答思路：

- synchronized 是 JVM 关键字，ReentrantLock 是 API 类
- 功能对比：可中断、超时、公平锁、多条件变量（Condition）

- 性能：JDK 6 后 synchronized 优化（偏向锁 → 轻量级 → 重量级）

✅ 示例回答：

synchronized 是 JVM 内置关键字，使用简单但功能有限；ReentrantLock 是 java.util.concurrent 提供的类，支持尝试获取锁（tryLock）、设置超时、响应中断、实现公平锁，并可通过 Condition 实现更精细的线程通信。在高竞争场景下，ReentrantLock 更灵活，但 synchronized 在低竞争下性能更好且自动释放锁，不易出错。

3. volatile 关键字的作用？能保证原子性吗？

回答思路：

- 作用：可见性 + 禁止指令重排序
- 不保证原子性（如 i++ 仍需 synchronized 或 AtomicInteger）
- 底层：内存屏障 + MESI 缓存一致性协议

✅ 示例回答：

volatile 主要解决两个问题：一是保证变量修改对其他线程立即可见，二是禁止 JVM 和 CPU 的指令重排序。但它不能保证复合操作的原子性，比如 count++ 实际包含读-改-写三步，即使 count 是 volatile，多线程下仍可能出错。此时应使用 AtomicInteger 或加锁。

二、JVM & 性能调优

4. JVM 内存结构？哪些区域会发生 OOM？

回答思路：

- 分代：堆（新生代、老年代）、方法区（元空间）、虚拟机栈、本地方法栈、程序计数器
- OOM 常见于：堆（对象太多）、元空间（加载类过多）、栈（递归太深）

✅ 示例回答：

JVM 内存分为线程共享区（堆、方法区）和线程私有区（栈、PC 寄存器等）。

- 堆：存放对象实例，OOM 最常见原因（如内存泄漏）
- 元空间（Metaspace）：存放类元数据，若动态生成大量类（如 CGLib）可能 OOM
- 虚拟机栈：每个方法调用创建栈帧，递归过深会导致 StackOverflowError（不是 OOM）
可通过 -Xmx、-XX:MaxMetaspaceSize 调整大小，并用 MAT 分析堆 dump。

5. 如何排查线上 CPU 100%？

回答思路：

- top → 找到高 CPU 进程 PID

- `top -Hp PID` → 找到高 CPU 线程 TID
- `jstack PID > thread.txt` → 转十六进制 TID 查找对应线程
- 分析代码（死循环、正则回溯、频繁 Full GC）

✅ 示例回答：

我会先用 `top` 查看哪个 Java 进程 CPU 高，再用 `top -Hp` 找出具体线程 ID。将线程 ID 转为 16 进制（如 12345 → 0x3039），然后用 `jstack`<PID>` 输出线程堆栈，在其中搜索该 16 进制 ID，定位到具体代码行。常见原因包括死循环、正则表达式灾难性回溯、或频繁 Full GC 导致 Stop-The-World。

三、Spring & Spring Boot

6. Spring Bean 的生命周期？

回答思路：

- 实例化 → 属性注入 → Aware 接口 → BeanPostProcessor 前后处理 → 初始化方法 → 使用 → 销毁

✅ 示例回答：

Spring Bean 生命周期大致分五步：

1. 通过构造器或工厂方法实例化
2. 属性填充（依赖注入）
3. 如果实现 `BeanNameAware`、`ApplicationContextAware` 等接口，调用对应方法
4. 执行 `BeanPostProcessor.postProcessBeforeInitialization`
5. 调用 `@PostConstruct` 或 `init-method` 指定的初始化方法
6. 执行 `BeanPostProcessor.postProcessAfterInitialization`
7. Bean 就绪，可被使用
8. 容器关闭时，调用 `@PreDestroy` 或 `destroy-method`

7. @Transactional 失效的常见场景？

回答思路：

- 自调用（`this.method()`）
- 非 public 方法
- 异常被捕获未抛出
- 数据库不支持事务（如 MyISAM）
- 传播行为配置错误

✅ 示例回答：

最常见的失效场景是类内部自调用。因为 Spring AOP 基于代理，只有通过代理对象调用带 `@Transactional` 的方法才会生效。如果在一个类中 `this.methodB()` 调用另一个 `@Transactional` 方

法，实际绕过了代理，事务不会开启。解决办法是注入自己（通过 ApplicationContext ）或拆分类。此外，异常被 try-catch 吞掉、方法非 public、或使用不支持事务的存储引擎也会导致失效。

四、数据库 & MyBatis

8. MySQL 索引失效的常见情况？

回答思路：

- 对字段使用函数或表达式（如 `WHERE YEAR(create_time) = 2023` ）
- 隐式类型转换（字符串字段传数字）
- LIKE 以通配符开头（`LIKE '%abc'` ）
- OR 条件中部分字段无索引
- 复合索引未遵循最左前缀

✅ 示例回答：

比如我们有一个 (name, age) 的联合索引。如果查询写成 `WHERE age = 20` ，就无法使用索引，因为违反了最左前缀原则。又比如 `WHERE name LIKE '%张'` ，由于前导通配符，索引无法用于快速定位。还有像 `WHERE name = 123` （name 是 varchar 类型），MySQL 会做隐式转换，导致索引失效。建议用 `EXPLAIN` 分析执行计划确认是否命中索引。

9. MyBatis #{} 和 \${} 的区别？

回答思路：

- #{} ：预编译占位符（PreparedStatement），防 SQL 注入
- \${} ：字符串替换，直接拼接，有注入风险

✅ 示例回答：

#{} 会被 MyBatis 转换为 JDBC 的 ? 占位符，通过 PreparedStatement 设置参数，能有效防止 SQL 注入，适用于绝大多数场景。而 \${} 是直接字符串替换，比如动态表名 `SELECT * FROM ${tableName}` ，但必须确保输入可信，否则极易被注入。因此，除非必要（如动态表名、列名），否则一律使用 #{} 。

五、系统设计 & 场景题

10. 如何设计一个分布式 ID 生成器？

回答思路：

- 方案对比：UUID（无序）、数据库自增（性能差）、Redis INCR（单点）、雪花算法（Snowflake）

- 重点讲 Snowflake：时间戳 + 机器 ID + 序列号
- 问题：时钟回拨（解决方案：等待、拒绝、用历史时间）

✅ 示例回答：

我推荐使用改进版雪花算法。ID 由 64 位组成：1 位符号位（0）+ 41 位时间戳（毫秒）+ 10 位机器 ID（可配置）+ 12 位序列号（每毫秒最多 4096 个）。优点是全局唯一、趋势递增、高性能。针对时钟回拨问题，可记录上次时间，若检测到回拨则短暂 sleep 或抛异常。我们项目中用的是美团的 Leaf-Segment 模式，兼顾性能与可用性。

11. 缓存穿透、击穿、雪崩的区别与解决方案？

回答思路：

- 穿透：查不存在的数据 → 布隆过滤器 + 空值缓存
- 击穿：热点 key 过期瞬间大量请求 → 互斥锁 / 永不过期
- 雪崩：大量 key 同时过期 → 随机过期时间 + 高可用架构

✅ 示例回答：

- 缓存穿透：恶意查询不存在的 ID，绕过缓存直打 DB。我们用布隆过滤器拦截非法 ID，并对空结果也缓存 1~2 分钟。
- 缓存击穿：如热门商品详情页缓存过期，瞬间高并发打垮 DB。我们采用“逻辑过期 + 后台异步更新”策略，或在重建缓存时加 Redis 分布式锁。
- 缓存雪崩：大量 key 同时失效。我们在设置 TTL 时加随机值（如 30 ± 5 分钟），并部署多级缓存（本地 + Redis）提高容灾能力。

六、项目经验（STAR 法则）

12. 请介绍一个你最有挑战的项目？

回答模板（STAR）：

- Situation：背景（电商大促、日活百万）
- Task：你的职责（负责订单模块重构）
- Action：你做了什么（引入 RocketMQ 解耦、Redis 缓存库存、分布式锁防超卖）
- Result：量化结果（QPS 从 500 提升到 3000，0 超卖）

✅ 示例简述：

我们在做秒杀系统时，面临高并发下库存超卖问题。我负责设计扣库存方案：前端限流 + Redis 预减库存 + RabbitMQ 异步下单 + MySQL 最终一致性校验。通过 Lua 脚本保证 Redis 扣减原子性，并用分布式锁防止重复提交。上线后支撑 5000 QPS，0 超卖，错误率 $< 0.1\%$ 。

七、编码题（常考）

13. 手写一个线程安全的单例模式

✔ 推荐写法（双重检查锁 + volatile）：

```
public class Singleton {
    private static volatile Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

注意：volatile 防止指令重排序导致其他线程拿到未初始化完成的对象。

总结：面试准备建议

维度	建议
基础	深挖 HashMap、线程、JVM、GC
框架	理解 Spring 循环依赖、AOP、事务原理
数据库	索引、事务、隔离级别、慢 SQL 优化
中间件	Redis 持久化/集群、MQ 可靠性
系统设计	能讲清楚 CAP、缓存策略、ID 生成
项目	用 STAR 法则，突出技术难点与结果

如果你希望我针对某个方向（如微服务、高并发、金融系统）提供更垂直的面试题，也可以告诉我，我可以进一步定制。祝你面试顺利，拿下 Offer！

user

提供相关的笔试的内容和详细的解答思路和内容

当然可以！Java 中级开发岗位的笔试通常包括以下几类题型：

- 选择题 / 判断题（考察基础概念）
- 填空题（考察 API、配置细节）
- 代码阅读题（考察对并发、异常、集合等的理解）
- 编程题（手写算法或业务逻辑）
- 简答题 / 设计题（考察系统思维）

下面我为你整理一份典型 Java 中级笔试内容 + 详细解答思路与参考答案，覆盖高频考点，助你高效备考。

一、选择题（示例 + 解析）

题目 1

以下关于 `synchronized` 的说法，错误的是？

- A. 可以修饰静态方法
- B. 可以修饰代码块
- C. 锁对象是当前实例（`this`）
- D. 被 `synchronized` 修饰的方法可以被多个线程同时访问

✅ 答案：D

🔍 解析：

- A 正确：静态方法锁的是 `Class` 对象
- B 正确：`synchronized(obj) { ... }`
- C 正确：非静态方法默认锁 `this`
- D 错误：同一把锁下，只能有一个线程进入

题目 2

JDK 8 中，以下哪个接口的实现类是线程安全的？

- A. `ArrayList`
- B. `HashMap`
- C. `ConcurrentHashMap`
- D. `StringBuilder`

✅ 答案：C

🔍 解析：

- `ArrayList`、`HashMap` 非线程安全
- `StringBuilder` 非线程安全（`StringBuffer` 才是）

- ConcurrentHashMap 是 JUC 提供的线程安全 Map

二、代码阅读题（理解执行结果）

题目 3

```
public class Test {  
    public static void main(String[] args) {  
        Integer a = 127;  
        Integer b = 127;  
        Integer c = 128;  
        Integer d = 128;  
        System.out.println(a == b); // ?  
        System.out.println(c == d); // ?  
    }  
}
```

✓ 输出：

```
true  
false
```

🔍 解答思路：

- Java 对 $-128 \sim 127$ 的 Integer 做了缓存（IntegerCache）
 - `==` 比较的是引用地址
 - a 和 b 指向同一个缓存对象 → true
 - c 和 d 超出范围，new 出两个不同对象 → false
- ⚠ 面试/笔试常考！强调：包装类比较应使用 `.equals()`

题目 4（多线程）

```
public class Counter {  
    private int count = 0;  
  
    public void increment() {  
        count++;  
    }  
  
    public int getCount() {  
        return count;  
    }  
  
    public static void main(String[] args) throws InterruptedException {  
        Counter c = new Counter();  
        Thread t1 = new Thread(() -> {
```

```

        for (int i = 0; i < 1000; i++) c.increment();
    });
    Thread t2 = new Thread() -> {
        for (int i = 0; i < 1000; i++) c.increment();
    };
    t1.start(); t2.start();
    t1.join(); t2.join();
    System.out.println(c.getCount());
}
}

```

✅ 可能输出：小于 2000 的任意值（如 1985、1992 等）

🔍 解答思路：

- count++ 不是原子操作（读 → 加1 → 写）
- 多线程并发修改导致竞态条件（Race Condition）
- 正确做法：加 synchronized 或使用 AtomicInteger

三、编程题（手写代码）

题目 5：实现一个线程安全的 LRU 缓存（要求支持 get/put）

✅ 考察点：

- 数据结构设计（哈希表 + 双向链表）
- 并发控制（ConcurrentHashMap + 锁 or 使用 Collections.synchronizedMap）
- LRU 淘汰策略

🔍 解答思路：

1. 使用 LinkedHashMap 可快速实现 LRU（但非线程安全）
2. 若需线程安全，可在外层加 synchronized，或使用 ConcurrentHashMap + 自定义双向链表（复杂）
3. 笔试中通常接受“加 synchronized”的简化版

✅ 参考答案（简化版）：

```

import java.util.LinkedHashMap;
import java.util.Map;

public class LRUCache`<K, V>` {
    private final int capacity;
    private final Map`<K, V>` cache;

    public LRUCache(int capacity) {
        this.capacity = capacity;
        this.cache = new LinkedHashMap`<K, V>`(16, 0.75f, true) {
            @Override
            protected boolean removeEldestEntry(Map.Entry`<K, V>` eldest) {
                return size() > LRUCache.this.capacity;
            }
        };
    }
}

```

```

    }
    };
}

public synchronized V get(K key) {
    return cache.getDefault(key, null);
}

public synchronized void put(K key, V value) {
    cache.put(key, value);
}
}

```

💡 如果题目明确要求“高并发”，可说明：“实际生产中会用 Caffeine 或 Redis 实现，避免自研”。

题目 6：反转链表（LeetCode Easy，但常考）

✅ 参考答案（迭代法）：

```

class ListNode {
    int val;
    ListNode next;
    ListNode(int x) { val = x; }
}

public ListNode reverseList(ListNode head) {
    ListNode prev = null;
    ListNode curr = head;
    while (curr != null) {
        ListNode next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}

```

🔍 关键点：

- 画图理解指针变化
- 注意边界（head 为 null）

四、简答题（原理 & 设计）

题目 7：Spring Boot 自动配置原理是什么？

✅ 参考答案：

Spring Boot 的自动配置基于 `@EnableAutoConfiguration` 注解。

其核心机制是：

1. 启动时扫描 `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` （旧版是 `spring.factories` ）
2. 加载所有 `XXXAutoConfiguration` 类（如 `DataSourceAutoConfiguration` ）
3. 这些配置类通过 `@ConditionalOnClass` 、 `@ConditionalOnMissingBean` 等条件注解，按需生效
4. 最终将 Bean 注入容器
例如：只要 classpath 有 HikariCP 和 `@EnableAutoConfiguration` ，就会自动配置 `DataSource`。

题目 8：MyBatis 如何防止 SQL 注入？

✅ 参考答案：

MyBatis 主要通过 `#{} 占位符` 实现防注入：

- `#{} 占位符` 会被编译为 JDBC 的 `PreparedStatement` 的 `?` 占位符
- 参数通过 `setString()` 、 `setInt()` 等方法传入，数据库将其视为纯数据而非 SQL 代码而 `${}` 是直接字符串拼接，存在注入风险，仅用于动态表名、列名等特殊场景，且需严格校验输入。
因此，业务参数一律使用 `#{} 占位符` 是防注入的最佳实践。

五、SQL 题

题目 9：查询每个部门工资最高的员工（含姓名、部门、工资）

假设有表：

- `employee(id, name, dept_id, salary)`
- `department(id, name)`

✅ 参考答案（窗口函数，MySQL 8.0+）：

```
SELECT e.name, d.name AS dept_name, e.salary
FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn
    FROM employee
) e
JOIN department d ON e.dept_id = d.id
WHERE e.rn = 1;
```

✅ 兼容低版本 MySQL（子查询）：

```
SELECT e1.name, d.name AS dept_name, e1.salary
FROM employee e1
```

```
JOIN department d ON e1.dept_id = d.id
WHERE e1.salary = (
    SELECT MAX(e2.salary)
    FROM employee e2
    WHERE e2.dept_id = e1.dept_id
);
```

⚠️ 注意：若同部门多人并列最高工资，子查询写法会返回多个，窗口函数可通过 `RANK()` 保留全部。

六、附加：常见陷阱题

题目 10：以下代码是否抛异常？

```
List<String> list = Arrays.asList("a", "b");
list.add("c");
```

- ✅ 答案：抛 `UnsupportedOperationException`
- 🔍 原因：`Arrays.asList()` 返回的是 `Arrays.ArrayList`（内部类），不支持 `add/remove` 操作。
- ✅ 正确做法：

```
List<String> list = new ArrayList<>(Arrays.asList("a", "b"));
```

总结：笔试准备建议

题型	备考重点
选择/判断	Java 基础、集合、并发、JVM、Spring 原理
代码阅读	包装类缓存、多线程竞态、异常处理、Stream 流
编程题	链表、字符串、数组、简单设计模式（单例、工厂）
SQL	JOIN、GROUP BY、窗口函数、索引优化
简答	Spring Boot、MyBatis、Redis、事务、缓存

🎯 最后建议：

- 刷 LeetCode Hot 100（重点：数组、链表、哈希、二叉树）
- 手写高频代码：单例、LRU、生产者消费者、线程池提交任务
- 熟悉常用命令：`jps`，`jstack`，`EXPLAIN`，`git rebase`
- 项目复盘：能用技术术语描述你解决过的问题

如果你需要 PDF 版题库 或 某公司（如阿里、腾讯、字节）真题模拟卷，也可以告诉我，我可以帮你整理。

祝你笔试顺利，一举通关！

user

需要 PDF 版题库 或

assistant

太好了！虽然我无法直接生成或发送 PDF 文件（当前交互环境不支持文件上传/下载），但我可以为你整理一份结构清晰、可复制粘贴的 Markdown 格式题库内容，你只需：

1. 全选下方内容
2. 粘贴到本地文本编辑器（如 VS Code、Typora、Word）
3. 导出为 PDF 即可

Java 中级开发面试 & 笔试高频题库（PDF 可用版）

适用于：Java 中级工程师岗位准备

内容涵盖：核心 Java、JVM、并发、Spring、数据库、Redis、系统设计、编码题等

更新时间：2026年2月

一、Java 核心基础

1. HashMap 的底层原理？为什么线程不安全？

答：

JDK 8 中 HashMap 采用 数组 + 链表/红黑树 结构。当链表长度 ≥ 8 且数组长度 ≥ 64 时，链表转为红黑树。

线程不安全原因：

- JDK 7：多线程扩容可能导致环形链表，引发死循环
 - JDK 8：虽修复环形问题，但并发 put 仍可能造成数据覆盖
- 替代方案：ConcurrentHashMap（CAS + synchronized 锁桶头）

2. synchronized 和 ReentrantLock 的区别？

维度	synchronized	ReentrantLock
类型	JVM 关键字	API 类 (java.util.concurrent.locks)
功能	自动加锁/释放	支持 tryLock、超时、中断、公平锁
条件变量	仅 wait/notify	支持多个 Condition
性能	JDK 6 后优化极佳	高竞争下更灵活

3. volatile 能保证原子性吗？

答：不能。
volatile 仅保证 可见性 和 禁止指令重排序，但不保证复合操作（如 i++ ）的原子性。
✔ 正确做法：使用 AtomicInteger 或加锁。

二、JVM & 性能调优

4. JVM 内存结构及 OOM 场景

- 堆（Heap）：对象实例 → OOM 最常见（内存泄漏）
- 方法区 / 元空间：类元数据 → 动态代理/CGLib 过多导致 OOM
- 虚拟机栈：方法调用栈帧 → StackOverflowError（非 OOM）
- 程序计数器、本地方法栈：极少 OOM

5. 如何排查 CPU 100%？

步骤：

```
top                # 找高 CPU 进程 PID
top -Hp `<PID>`    # 找高 CPU 线程 TID
printf "%x\n" `<TID>`    # 转 16 进制（如 12345 → 3039）
jstack `<PID>` | grep -A 20 3039 # 定位代码行
```

常见原因：死循环、正则回溯、频繁 Full GC

三、Spring & Spring Boot

6. Spring Bean 生命周期

1. 实例化（构造器 / FactoryBean）
2. 属性注入（@Autowired）

- 3. Aware 接口回调 (BeanNameAware 等)
- 4. BeanPostProcessor.postProcessBeforeInitialization
- 5. @PostConstruct / init-method
- 6. BeanPostProcessor.postProcessAfterInitialization
- 7. 使用
- 8. @PreDestroy / destroy-method (容器关闭时)

7. @Transactional 失效场景

-  方法非 public
-  自调用 (this.method())
-  异常被捕获未抛出
-  数据库引擎不支持事务 (如 MyISAM)
-  传播行为配置错误 (如 REQUIRES_NEW 嵌套)

四、数据库 & MyBatis

8. MySQL 索引失效场景

-  对字段使用函数： WHERE YEAR(create_time) = 2023
-  隐式类型转换： WHERE name = 123 (name 是 varchar)
-  LIKE 以 % 开头： LIKE '%abc'
-  OR 条件中部分字段无索引
-  联合索引未遵循最左前缀

9. MyBatis #{} vs \${}

语法	机制	安全性	用途
#{}	PreparedStatement 占位符	 防 SQL 注入	普通参数
\${}	字符串直接拼接	 有注入风险	动态表名、列名

五、Redis & 缓存

10. 缓存三大问题

问题	原因	解决方案
穿透	查询不存在的数据	布隆过滤器 + 空值缓存 (TTL 短)

问题	原因	解决方案
击穿	热点 key 过期瞬间高并发	互斥锁 / 逻辑过期 + 后台更新
雪崩	大量 key 同时过期	随机 TTL (如 30±5 分钟) + 高可用

六、系统设计

11. 分布式 ID 生成方案

- UUID：无序，不适合作为主键
- 数据库自增：性能差，单点
- Redis INCR：依赖 Redis，需持久化
- 雪花算法（Snowflake）：
1bit(符号) + 41bit(时间戳) + 10bit(机器ID) + 12bit(序列号)
 注意：时钟回拨问题（ 解决方案：等待、拒绝、历史时间池 ）

七、编程题（手写）

12. 线程安全单例（双重检查锁）

```
public class Singleton {
    private static volatile Singleton instance;
    private Singleton() {}
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

13. 反转链表

```
public ListNode reverseList(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode next = curr.next;
        curr.next = prev;
        prev = curr;
    }
}
```

```
    curr = next;
  }
  return prev;
}
```

14. LRU 缓存（简化版）

```
public class LRUCache`<K, V>` {
    private final int capacity;
    private final Map`<K, V>` cache;

    public LRUCache(int capacity) {
        this.capacity = capacity;
        this.cache = new LinkedHashMap`<K, V>`(16, 0.75f, true) {
            protected boolean removeEldestEntry(Map.Entry`<K, V>` eldest) {
                return size() > LRUCache.this.capacity;
            }
        };
    }

    public synchronized V get(K key) { return cache.getDefault(key, null); }
    public synchronized void put(K key, V value) { cache.put(key, value); }
}
```

八、SQL 题

15. 查询各部门工资最高员工

```
-- MySQL 8.0+
SELECT e.name, d.name AS dept, e.salary
FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) rn
    FROM employee
) e
JOIN department d ON e.dept_id = d.id
WHERE rn = 1;
```

九、陷阱题

16. Arrays.asList() 能 add 吗？

答：不能！

返回的是 Arrays.ArrayList（内部类），不支持 add/remove，会抛 UnsupportedOperationException。

✓ 正确写法：

```
List`<String>` list = new ArrayList<>(Arrays.asList("a", "b"));
```

十、项目经验 STAR 模板

Situation：背景（如“大促期间订单系统压力剧增”）

Task：你的任务（“负责订单创建模块性能优化”）

Action：技术动作（“引入 RocketMQ 异步解耦，Redis 预减库存，分布式锁防超卖”）

Result：量化结果（“QPS 从 800 提升至 5000，0 超卖，错误率 < 0.1%”）

✓ 使用说明：

- 将本内容复制到 [Typora](#)、VS Code（安装 Markdown 插件）或 Word
- 调整字体/标题样式后，导出为 PDF
- 建议打印或分章节背诵，配合 LeetCode 刷题

如果你希望我：

- 补充 微服务（Spring Cloud）专题
- 添加 消息队列（Kafka/RocketMQ）题库
- 提供 某公司真题模拟卷（如阿里 P6 级别）

请直接告诉我，我可以继续为你定制！

祝你求职顺利，Offer 收割不断！🎯